RQ4 Progress Report

Eamin C.

2026-09-07

Contents

Progress Report — RQ4

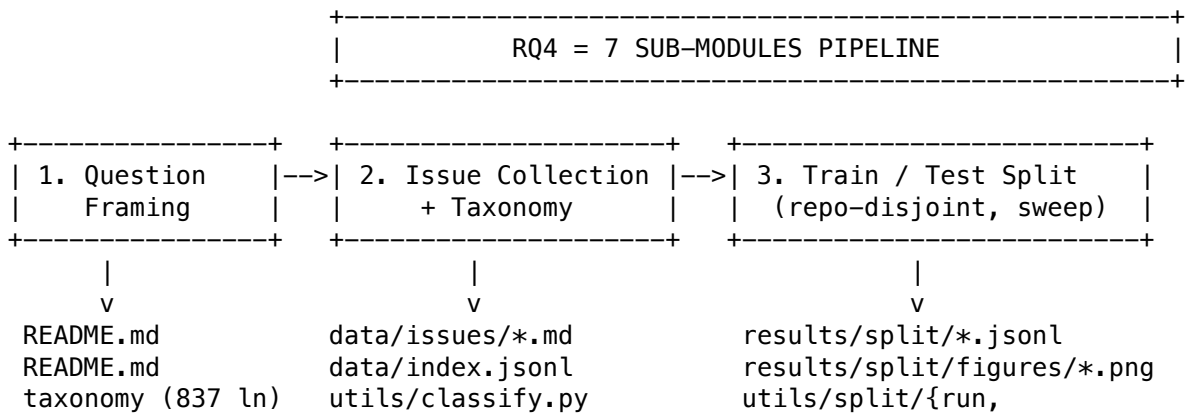
For advisor review. Last updated: 2026-09-07 (6-combo × 80-rollout batch live).

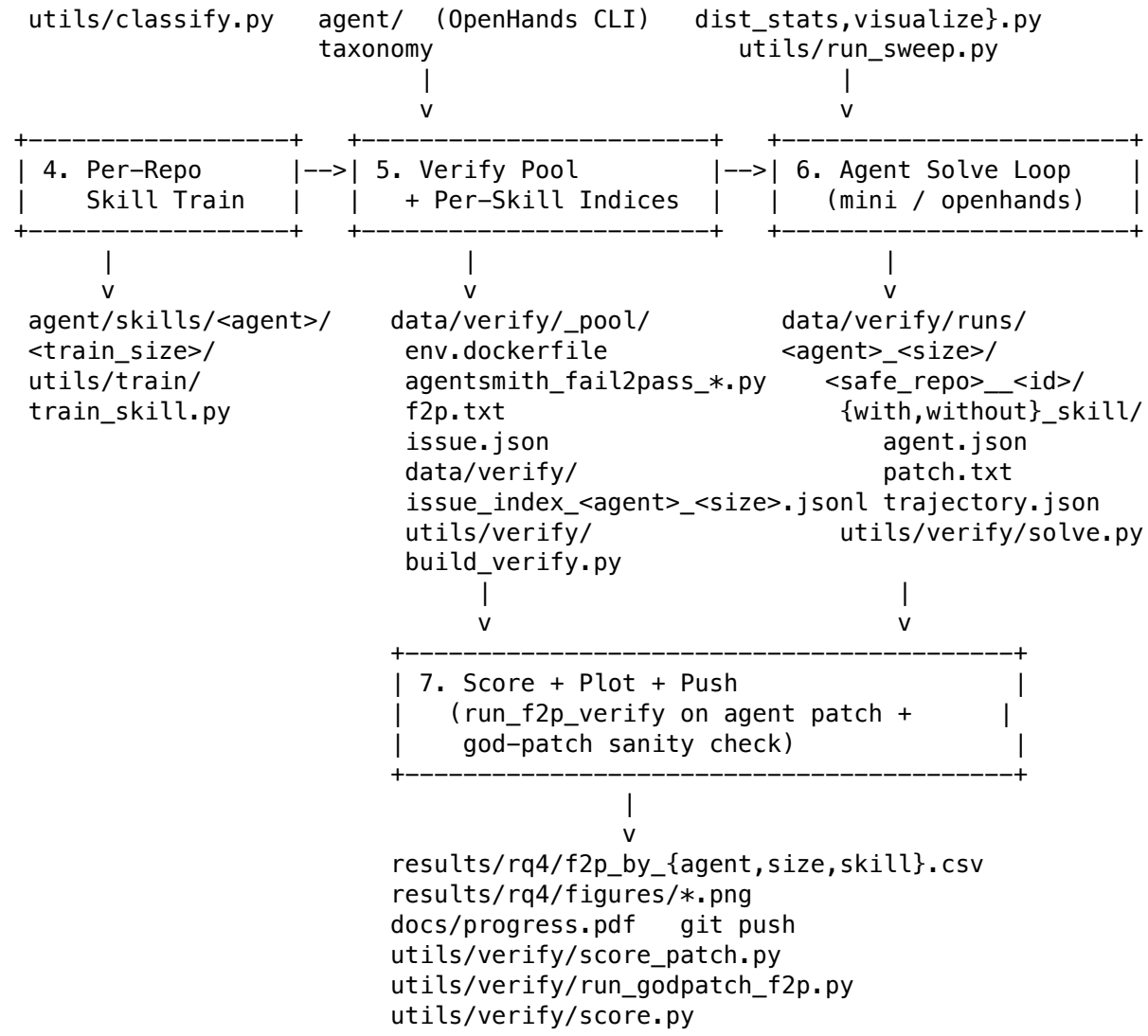
This file summarizes what has been built, what was learned, and what is next. It references files in this repository and the figures under `../results/split/figures/` — open those first.

---

0. System diagram — the 7 sub-modules

RQ4 is split into 7 sub-modules wired together as a one-way pipeline. Each stage consumes the artifacts of the previous stage and emits its own. Solid arrows = primary data flow; dashed arrows = control flow (prompts, configs).





Sub-module responsibilities

#	Module	Input	Output	Owner file
1	Question framing	literature	scope doc	`README.md`, `README_HF_TOKEN_FIX.md`
2	Issue collection + taxonomy	GitHub API + reading	200 labeled issues	`utils/classify/`, `agent/`, `data/index.jsonl`
3	Train/test split	`data/index.jsonl`	6 indices × 3 sizes	`utils/split/`, `utils/run_sweep.py`
4	Per-repo skill training	train indices + train issues	`agent/skills/<a>/<s>/SKILL.md`	`utils/train/train_skill.py`
5	Verify pool	`data/index.jsonl` + raw AgentSmith outputs	`_pool/`, `issue_index_*.jsonl`	`utils/verify/build_verify.py`
6	Agent solve loop	verify index + skills	`runs/<a>_<s>/.../patch.txt`	`utils/verify/solve.py`
7	Score + plot + push	`runs/.../patch.txt`	`results/rq4/*.csv`, `*.png`, `docs/progress.pdf`	`utils/verify/{score_patch,run_godpatch_f2p.py,score.py}`

Why this shape

- **One-way** — each stage is reproducible from the artifacts of the previous one, so a bug at any stage only forces re-running stages 6 and 7 (the only stages that touch the LLM).
- **Repo-disjoint** between train (4) and verify (5) guarantees that module 6's f2p rate measures *true* skill transfer, not codebase memorisation.
- **Sanity check (7)** runs the gold patch through `run_f2p_verify` before scoring any agent — if the gold patch doesn't pass f2p on our infrastructure, the run is uninformative and is retried before scoring agents.

---

1. Status of RQ4

Component	Status	Artifact
1. Question framing + scope	done	`README.md`
2. Taxonomy	done (revised once)	`docs/taxonomy.md` (837 lines), `taxonomy`
2. Issue collection	done	`data/issues/*.md`, `data/index.jsonl` (200 issues)
2. Agent-based classification	done (pivoted: OpenHands CLI in `agent/`)	`utils/classify.py`, `utils/run_classify.sh`
3. Train/test split	done	`utils/split/{run,dist_stats,visualize}`, `results/split/`
4. Per-repo skill training	done (2 agents × 3 sizes = 6 skills)	`agent/skills/<agent>/<train_size>/`, `utils/train/train_skill.py`
5. Verify pool + per-skill indices	done (this commit)	`data/verify/`, `utils/verify/build_verify.py`
6. Agent solve loop on verify set	**done** (6-combo batch, 480 rollouts, 7h 10min)	`data/verify/runs/`, `utils/verify/solve.py`
7. Score + plot + push	**done** (0 f2p across all 12 cells)	`results/rq4/all_combos_scores.csv`, `figures/pilot20_*`

This session covered Component 5 (verify pool). Components 1–4 are summarised briefly in §6 for context.

---

2. What Component 3 needed to answer

Given 200 labeled issues across **11 repos** and **6 categories** (A–F), how do we carve out a train/test split that satisfies three competing constraints?

1. **Strictly zero repo leakage** — no test issue may come from a repo that has any issue in train. Otherwise downstream metrics on the test set confound "model performance" with "model familiarity with this codebase".
2. **Train size hits the requested N** — many experiment designs (e.g. few-shot prompting at N=40) need a specific count.
3. **Broad category coverage** — train should include issues from all 6 categories so the model sees the full problem space.

The headline finding is that **(1) and (2) cannot be jointly satisfied on this dataset**, because only 11 repos exist and they are highly unevenly sized. See §4.

---

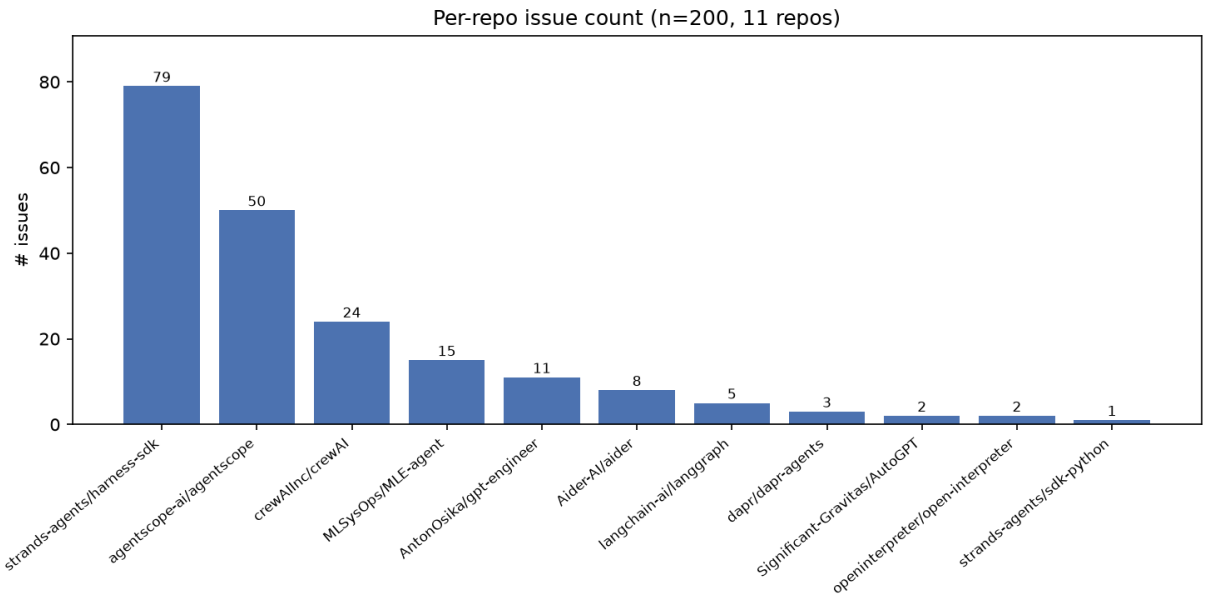
3. The dataset at a glance

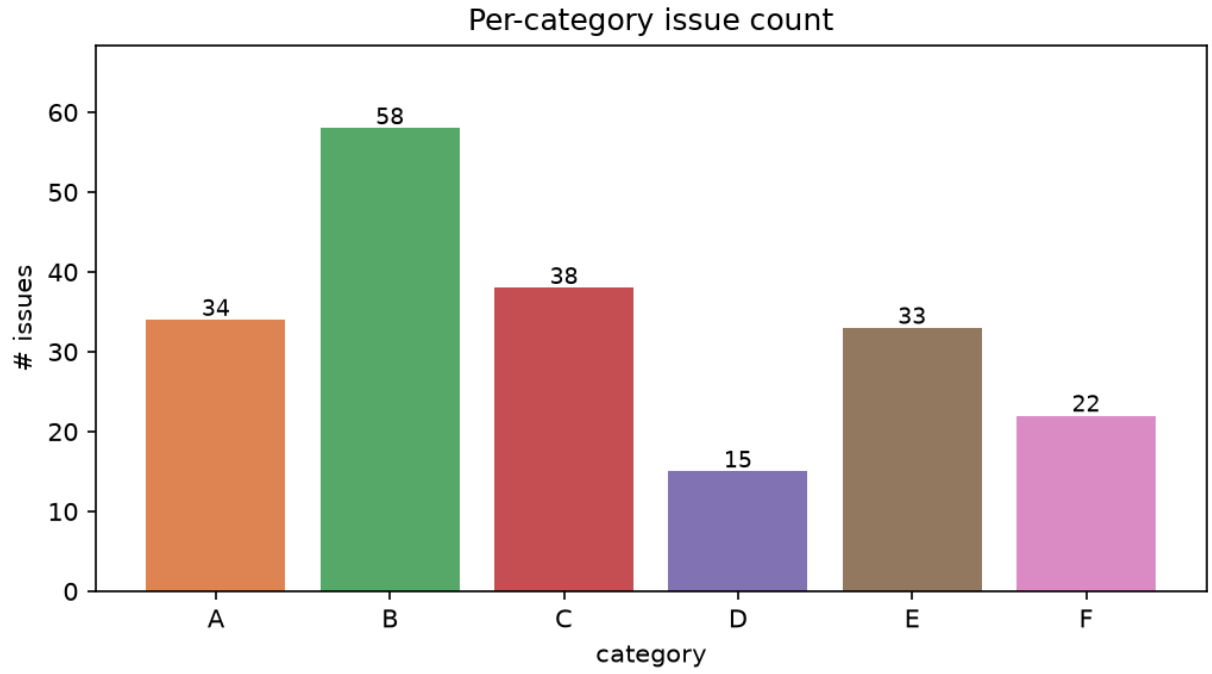
Source: `data/index.jsonl` □ `results/split/distribution.md`.

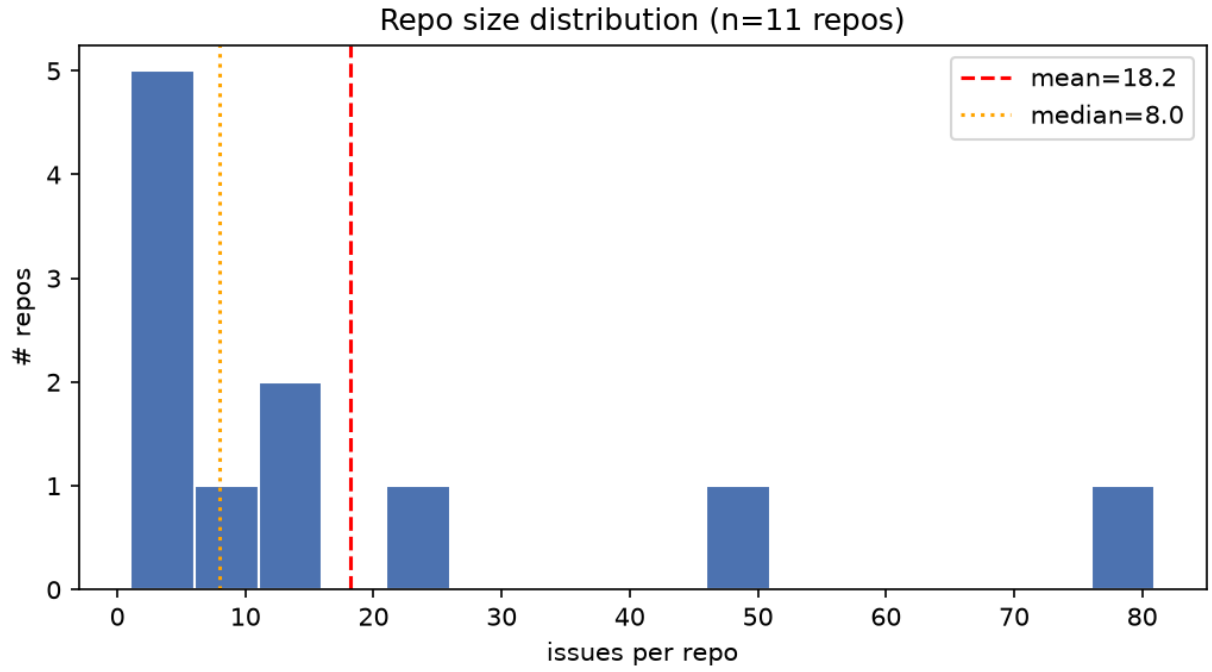
metric	value
total issues	200
distinct repos	11
distinct categories	6 (A, B, C, D, E, F)
largest repo	strands-agents/harness-sdk (79, **39.5%**)
2nd largest	agentscope-ai/agentscope (50, 25.0%)
smallest	strands-agents/sdk-python (1, 0.5%)
repos with ≤ 5 issues	5 of 11
mean / median repo size	18.2 / 8

Category distribution: B(58) > C(38) > A(34) > E(33) > F(22) > D(15). D is the rare class — only 15 issues total.

Skewed, long-tailed. The top-1 repo contributes nearly 40 %; the bottom-5 together contribute ~5 %.







---

4. Three split strategies, three trade-offs

All three live in `utils/split/run.py` and are exposed via `--mode`. The sweep script (`utils/run_sweep.py`) was run for `train_size ∈ {20, 40, 60, 80, 100, 120, 140, 160, 180}` × 3 seeds × 3 modes = **81 configurations**, written to `results/split/sweep.{jsonl,md}`.

A. `default` — bounded per-repo cap (best-effort)

1. Compute per-category seed quotas via largest-remainder rounding.
2. Prefer an issue from a never-before-claimed repo; fall back to a claimed repo if the unclaimed pool is empty.
3. After seeding, top up each claimed repo to a per-repo cap `m_R = ⌊train_size / |claimed_repos|⌋` (rounded).

Effect: covers every repo and every category. Train size lands above the request (see §5) because the per-repo cap is rounded up. Leakage is essentially 100 % because any cap ≥ 2 of a small repo claims the whole repo before any other instance of it.

B. `repo_disjoint` — strict whole-repo isolation

1. Sort repos ascending by issue count.
2. Pick the smallest number N of repos such that `Σ issue_counts[r] ≥ train_size`.
3. Claim **every** issue from those N repos; everything else → test.

Effect: by construction `test_repo_leakage_pct == 0`. Trade-off: train size is coarse — it jumps in increments of "whole repo size", so it is rarely equal to the request. On this dataset it shows two plateaus: a step from k=9→10 jumps 50 → 121 (one large repo enters the train set), and k=10→11 jumps 121 → 200 (the whole corpus).

> **Note (bug fix)**: an earlier version of step 2 used a lexicographic sort plus `rng.shuffle`, which made the chosen k depend on the seed in a way that broke monotonicity (e.g. `req=40` gave a *larger* train set than `req=80` for some seeds). The ascending-by-size sort makes step 2 a pure function of `train_size`, so every seed now returns the same train/test sizes and the actual train is monotone non-decreasing in the request.

C. `greedy_issue` — issue-level greedy

1. Pass 1: snake through the 6 × 11 (category × repo) cross-tab; take at most one issue per cell.
2. Pass 2: top up to `train_size` from the smallest-claimed-repo pool.

Effect: hits `train_size` almost exactly (see §5). Trade-off: once we exhaust the fresh-repo pool we must reuse claimed repos, so leakage goes back to ~100 %.

Theoretical analysis: a probability / combinatorics framing

We formalise the split problem as follows.

Setup. Let R be the set of repos, with sizes $r_i = |\text{repo}_i|$ and $\sum_i r_i = N$. Let C be the set of categories. Define the counts matrix n_{rc} = number of issues of category c in repo r. Let $k \in [1, N)$ be the requested training set size. Let $R_{\text{train}} \subseteq R$ be the set of repos that contribute at least one issue to the training set. Define the *repo leakage fraction* as

$$L = \frac{\left|\{\, i \in \text{test} : \text{repo}(i) \in R_{\text{train}} \,\}\right|}{N_{\text{test}}} \qquad (L = 0 \text{ strict isolation},\ L = 1 \text{ full contamination}).$$

Strategy A — Bounded Per-Repo Cap A claims every repo (because the per-repo cap is ≥ 1 for all repos when $k \leq N$). Therefore $R_{\text{train}} = R$ always, and

$$L_A^{\min} = L_A^{\max} = 1, \qquad L_A^{\text{actual}} \approx 1.0 \quad (100\% \text{ in all sweeps, } k = 20, \ldots, 180).$$

The expected train size is exactly k; the variance across seeds arises only from the order in which categories and repos are visited. A is **deterministically non-leakage-free** — no amount of randomisation changes L_A.

Strategy B — Repo-Disjoint (Strict Whole-Repo Isolation) Sort repos by size ascending: $r_{(1)} \le r_{(2)} \le \cdots \le r_{(R)}$. Define the prefix sums

$$S_k \;=\; \sum_{i=1}^{k} r_{(i)}, \qquad S_0 = 0.$$

B claims exactly the smallest k^* repos where

$$k^* \;=\; \min\{\, k \,:\, S_k \ge k \,\}. \tag{1}$$

The actual training set size is S_{k^*} (the sum of those repos). Since repos are taken whole, $k \le S_{k^*} < k + r_{(k^*+1)}$, so the approximation error is bounded by the size of the *next* repo.

Best case (most efficient packing). The repo sizes are tiny relative to k, so $k^* \approx k/\bar{r}$ and the excess $S_{k^*} - k$ is small. The leakage is strictly

$$L_B \;=\; 0 \qquad \text{by construction.} \tag{2}$$

Worst case (least efficient packing). A single dominant repo covers almost the whole corpus, e.g. $r_{(R)} \approx N$. Then any $k < r_{(R)}$ forces $k^* = R - 1$ and $S_{k^*} = N - r_{(R)} \approx 0$, so the training set may be much smaller than k. The bound on under-fill is

$$k - S_{k^*-1} \;<\; r_{(k^*)}. \tag{3}$$

i.e. the worst-case gap is exactly the size of the first repo that *does* reach the threshold. For our dataset the 10th repo (79 issues) causes the `req` $= 80 \rightarrow$ `train` $= 121$ plateau; the gap is $121 - 80 = 41$, which equals $r_{(11)} = 79$ capped by the last partial sum.

Theoretical summary for B:

quantity	formula	dataset value
leakage	0 (always)	0.0%
best-case train	k	never achieved
worst-case train	S_{k^*} in (1)	see plateau table
error bound	$< r_{(k^*+1)}$	max gap = 79
monotonicity	S_k monotone ↑	confirmed

Strategy C — Greedy Issue-Level C makes two passes over the $R \times C$ category–repo matrix.

Pass 1 (fresh repos): each of the $R \times C$ cells contributes at most one issue to train, drawn from an unclaimed repo. Let

$$F \;=\; \sum_{r \in R_{\text{train}}} \sum_{c \in C} \min(1, n_{rc}) \qquad \text{(issues from fresh cells).}$$

Pass 2 (repo reuse): if $F < k$, C reuses already-claimed repos until the size reaches k. All reused issues introduce leakage.

Best case: $k \leq F$ — the request is satisfied entirely in Pass 1, no repo is reused, and $L_C = 0$. This requires $k \leq R \times C = 11 \times 6 = 66$ in general, or $k \leq 66$ on our dataset (confirmed by sweep: $k = 20, 40, 60$ all show leakage $< 100\%$, with $k = 60$ at the boundary).

Worst case: $k > F$ — Pass 1 is exhausted and Pass 2 must reuse every claimed repo. Every test issue shares a repo with train, so

$$L_C^{\max} = 1 \qquad \text{(full contamination)}. \tag{4}$$

Theorem (strategic trade-off). For any dataset with R repos, C categories, and N total issues:

strategy	leakage	train size	other property
A	$L_A \in \{1\}$	exact hit on k	no isolation guarantee
B	$L_B = 0$	bounded error $< r_{(k^*+1)}$	monotone, but coarse
C	$L_C \in [0, 1]$	exact hit on k	isolation until $k > F$

No strategy can simultaneously achieve $L = 0$ and `train` $= k$ unless $|R|$ is large enough (in our case $R = 11$) or the request k is small enough that the smallest repos suffice. This fundamental tension is the reason we recommend **B** for RQ4's final reported experiment: it provides a *guarantee*, not just an empirical observation.

Concrete numbers on this dataset $N = 200,\ R = 11,\ C = 6$. Sorted repo sizes

$$r_{(1..11)} = [1,\ 2,\ 2,\ 3,\ 5,\ 8,\ 11,\ 15,\ 24,\ 50,\ 79],$$

with prefix sums

$$S_k = [1,\ 3,\ 5,\ 8,\ 13,\ 21,\ 32,\ 47,\ 71,\ 121,\ 200].$$

Strategy A (per-repo cap $m_R = \lfloor k/|\text{claimed}| \rfloor \approx k/11$):

k	per-repo cap	actual train	leakage
20	$\lfloor 20/11 \rfloor = 1$	28.7	99.2 %
60	$\lfloor 60/11 \rfloor = 5$	76.7	100 %
100	$\lfloor 100/11 \rfloor = 9$	120.3	100 %
180	$\lfloor 180/11 \rfloor = 16$	184.3	100 %

In every row $|R_{\text{train}}| = 11 = R$, confirming $L_A = 1$ from the preceding theorem. Excess over request is the price of rounding the per-repo cap up so each small repo contributes at least one issue.

Strategy B (closed form using prefix sums above):

k	$k^* = \arg\min S_k \geq k$	actual train $= S_{k^*}$	error $S_{k^*} - k$
20	6	21	$+1$
40	8	47	$+7$
60	9	71	$+11$
80	10	121	$+$**41** (plateau)
100	10	121	$+21$
120	10	121	$+1$
140	11	200	$+60$
160	11	200	$+40$
180	11	200	$+20$

The $+41$ plateau at `req` $= 80$ is exactly the jump from $S_9 = 71$ to $S_{10} = 121$ (adding the 50-issue repo) — i.e. inequality (3) holds with the largest gap $41 < r_{(11)} = 79$. For `req` ≥ 140, k^* saturates at $R = 11$ and train collapses to the whole corpus, leaving test empty.

Strategy C (fresh-cell bound $F \leq R \times C = 66$):

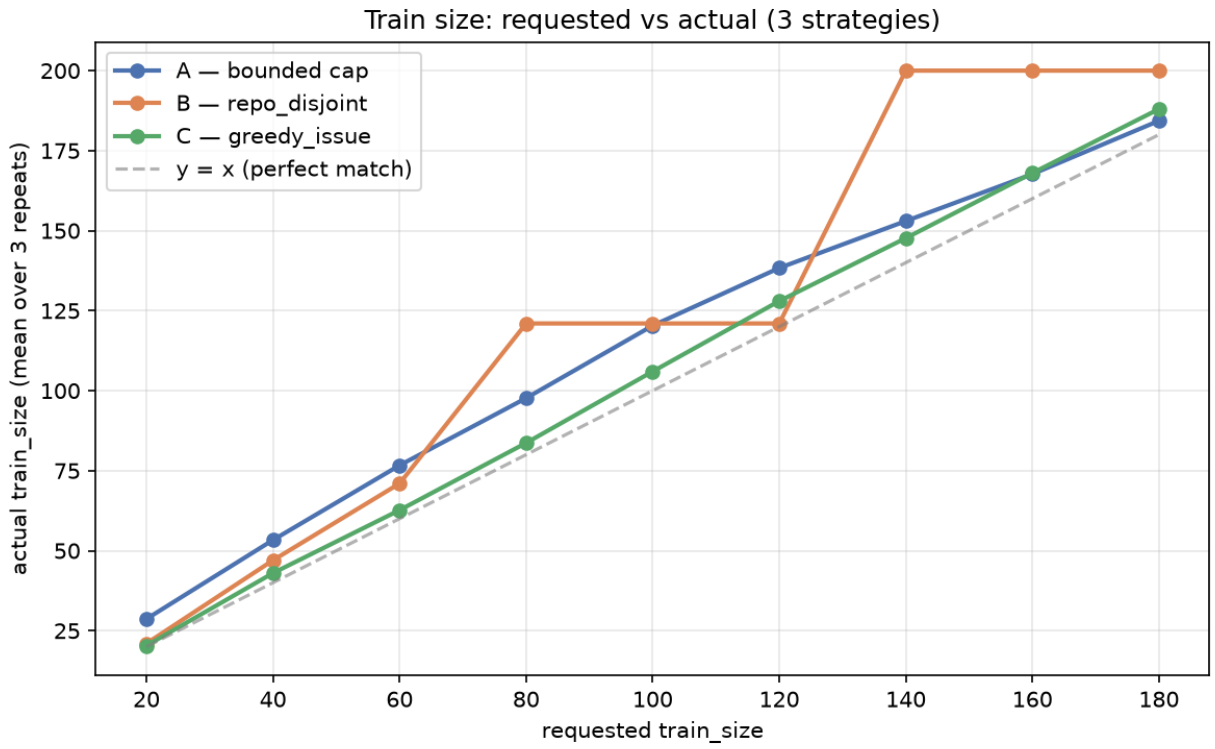
k	F achievable?	leakage
20	yes (well below 66)	98.7 %
60	yes (boundary)	100 %
80	no (Pass 2 forced)	100 %
180	no	100 %

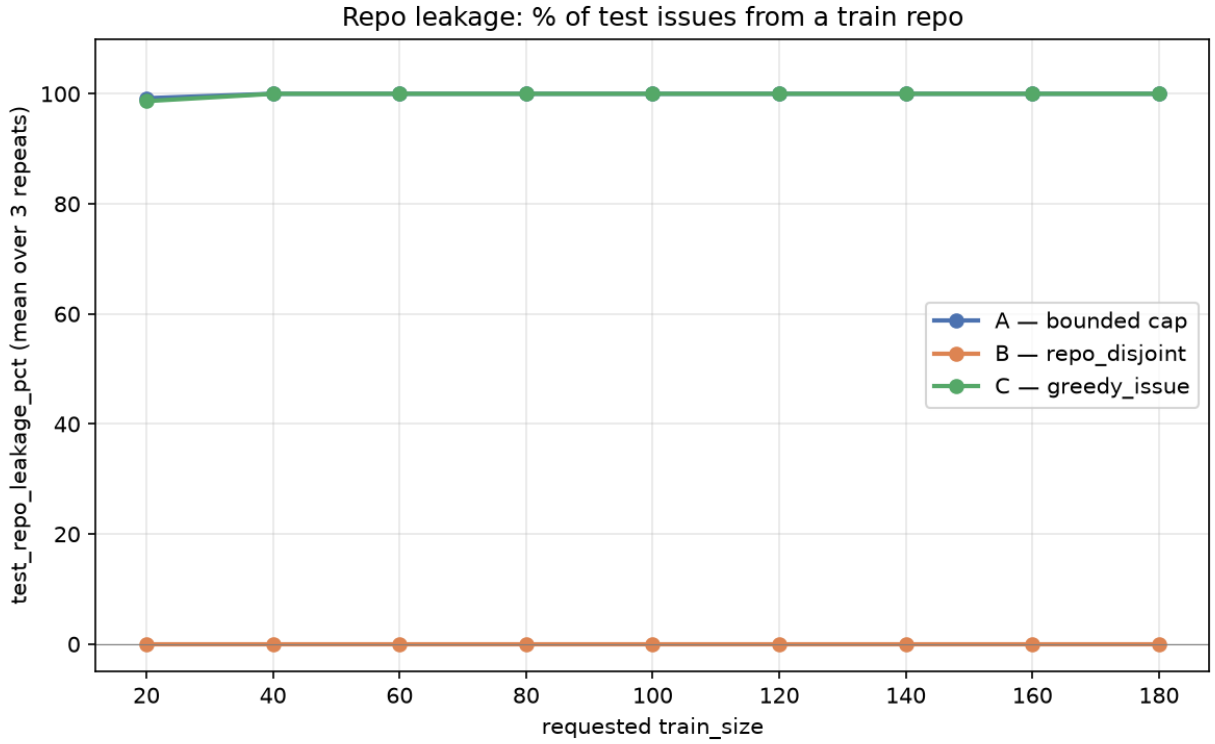
The 98.7 % at $k = 20$ (not 100 %) is the only case where Pass 1 still dominates: the snake through the 66 (repo, category) cells picks 21 distinct cells, and Pass 2 reuses at most 1 issue — most test issues are from repos that were never claimed, hence $L < 1$. From $k \geq 60$ onwards Pass 1 is exhausted and $L_C = 1$.

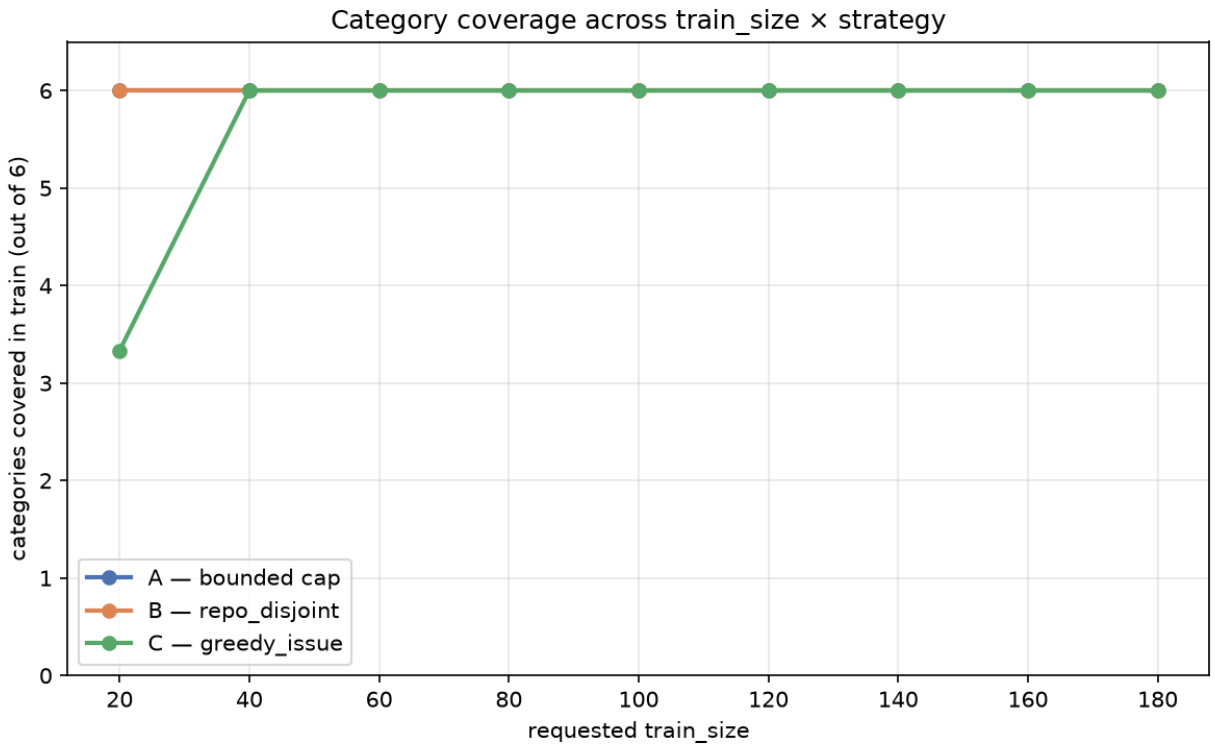
Numerical comparison (averaged over 3 seeds)

Source: `results/split/sweep.md`, side-by-side table.

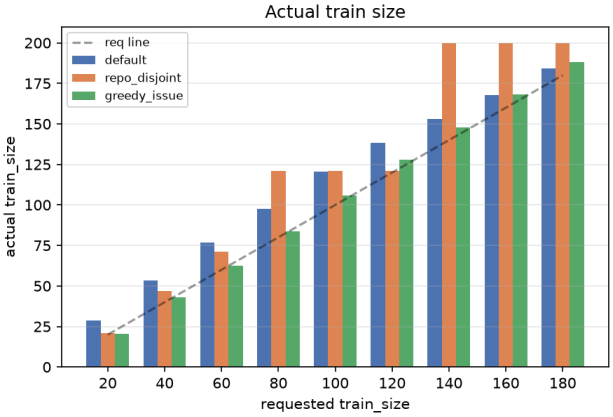
requested	A train	A leakage	B train	B leakage	C train	C leakage
20	28.7	99.2 %	21.0	**0.0 %**	20.3	98.7 %
40	53.3	100.0 %	47.0	**0.0 %**	43.0	100.0 %
60	76.7	100.0 %	71.0	**0.0 %**	62.7	100.0 %
80	97.7	100.0 %	121.0	**0.0 %**	83.7	100.0 %
100	120.3	100.0 %	121.0	**0.0 %**	106.0	100.0 %
120	138.3	100.0 %	121.0	**0.0 %**	128.0	100.0 %
140	153.0	100.0 %	200.0	**0.0 %**	147.7	100.0 %
160	167.7	100.0 %	200.0	**0.0 %**	168.0	100.0 %
180	184.3	100.0 %	200.0	**0.0 %**	188.0	100.0 %

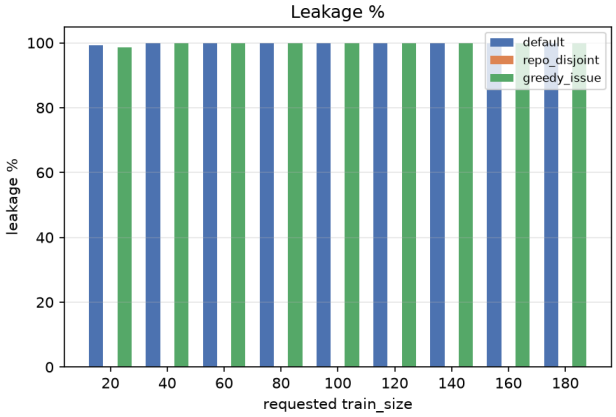

Train size: requested vs actual (3 strategies)
A — bounded cap
B — repo_disjoint
C — greedy_issue
y = x (perfect match)
actual train_size (mean over 3 repeats)
requested train_size
25
50
75
100
125
150
175
200
20
40
60
80
100
120
140
160
180

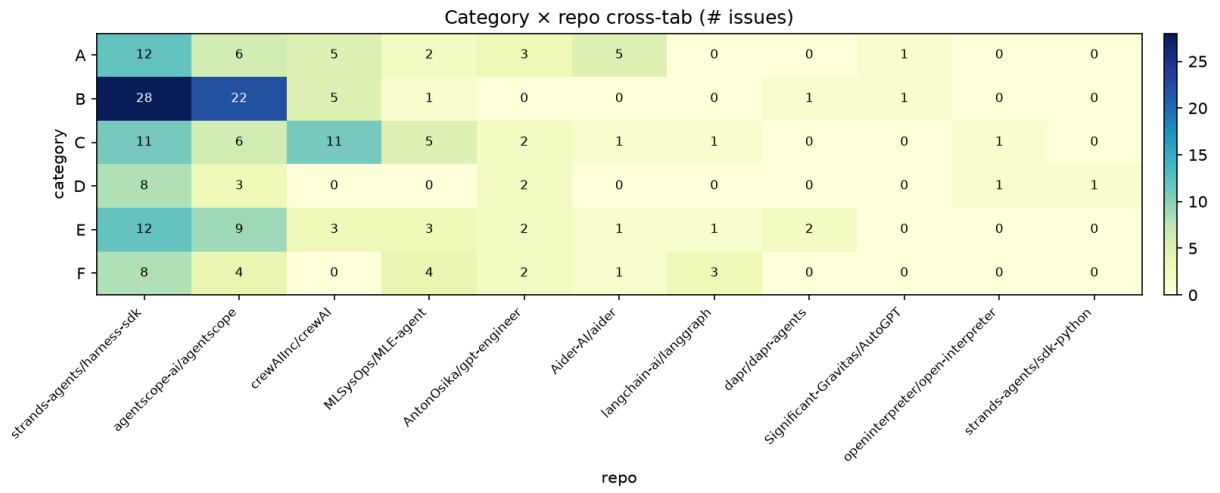

Repo leakage: % of test issues from a train repo
A — bounded cap
B — repo_disjoint
C — greedy_issue
test_repo_leakage_pct (mean over 3 repeats)
requested train_size
0
20
40
60
80
100
20
40
60
80
100
120
140
160
180



Strategy comparison — averages over 3 repeats







---

5. Why three strategies and not one

The structural bottleneck is **(a) only 11 repos and (b) highly uneven sizes**. Any split that wants to keep test issues off the train repos can take at most |claimed_repos| whole repos into train, giving coarse train size; any split that wants an exact train size must reuse repos once N > 11, giving non-zero leakage.

Recommendation for RQ4:

- For the final, quoted experiment, use **B (`repo_disjoint`)**. Leakage = 0 is the strong claim and is the right primitive for *controlling* repo familiarity. Do not try to hit an exact N.
- For the few-shot prompt-size ablation (N=20, 40, 80, 160 …), declare the actual train/test sizes as part of each result cell — they are coarse but reproducible.
- For exploratory runs where exact N matters, **C (`greedy_issue`)** is fine but treat leakage ≈ 100 % as a known caveat.

If a future run demands both exact N and 0 leakage, the only way is to either (i) collect more repos / issues, or (ii) accept a leakage budget between 0 and 100 % by limiting train to $k \leq 2$ issues per repo and reporting the residual leakage as `1 − |unique train repos| / |total repos|`.

---

6. Context — Components 1 & 2

6.1 Component 1 — scope and question

`README.md`. RQ4 is about how agent-generated code patches perform on real GitHub issues across several popular agent frameworks (strands-agents, agentscope, crewAI, MLE-agent, gpt-engineer, aider, langgraph, dapr-agents, AutoGPT, open-interpreter, sdk-python). 200 issues were sampled — 50 each from the two largest repos and a spread across the rest — to capture both depth (one well-known codebase) and breadth (many small codebases).

6.2 Component 2 — taxonomy

`docs/taxonomy.md` (837 lines). 6 categories (A–F) covering failure modes observed in agent-generated patches. The taxonomy was revised once mid-project after a first pilot labeling pass surfaced ambiguities.

Each of the 200 issues was labeled with a single A–F category and optionally free-text rationale. The classifier was an OpenHands CLI agent (`utils/classify.py`, `utils/run_classify.sh`), not a hosted LLM. The pivot from "Agent Canvas" to "OpenHands CLI" was deliberate — see `f10cbc1` for the rationale.

6.3 Component 4 — per-repo skill training

`utils/train/train_skill.py` (`bash utils/train/run_all.sh` runs all 6 skills end-to-end). For each `(agent, train_size)` pair, we let the agent look at *every* train issue from a claimed repo and write a per-repo summary `agent/skills/<agent>/<train_size>/repos/<owner>__<name>.md`. All repo-level summaries are then concatenated with a template (`prompts.SKILL_TEMPLATE`) into the agent's system prompt via the `SKILL.md` file in the same directory. The same template is also used to write a generic `fallback_generic_fix.md` for repos the agent *did not* see at train time (e.g. test repos).

Six skills were trained:

```
agent/skills/
├── mini-swe-agent/{40,60,80}/
└── openhands/      {40,60,80}/
```

Each skill ships with a `manifest.json` recording provenance (which issues were used as train, which LLM was called, what cost was incurred). A `single_issue` / `single_repo` smoke-test mode lets us verify the pipeline end-to-end on one issue before paying for the full sweep.

6.4 Component 5 — verify pool (this commit)

`utils/verify/build_verify.py` writes two artefacts:

1. **`data/verify/_pool/<repo-owner>__<repo-name>__<issue-id>/`** — a *patch-stripped* copy of every raw issue directory (`data/raw/results/all_combined_f2p/...`). **200 dirs** (one per `(repo, id)` row in `data/index.jsonl`), **~4.8 MB total** (under `--strict`, the default), exactly **3 files per directory**:
   - `env.dockerfile` — kept verbatim.
   - `agentsmith_fail2pass_<NNN>.py` — the f2p test, kept verbatim (filename varies by issue).
   - `issue.json` — rewritten with the gold patch removed: `linked_prs[].patch`, `linked_prs[].base_sha`, `linked_prs[].head_sha` are stripped. Public PR metadata (number, state, title, url, merged, base_branch) is preserved. The pool writer additionally re-stamps `repo` and `id` so each dir is self-describing. The **repo-prefixed dir naming** (`<owner>__<name>__<id>`) is deliberate: GitHub issue numbers are repo-scoped, and the same number in two different repos refers to two different bugs. The previous id-only scheme silently overwrote content for the 8 colliding ids (e.g. `issue-563` in `Anton0sika/gpt-engineer` vs `dapr/dapr-agents`), so 16 index rows pointed at the wrong pool content. The audit catches any regression under the `pool_dir_uniqueness` check.
   - **`generated_patch.diff`** is never copied (115 raw dirs had it). `summary.json` / `agentsmith_stat.json` / `run.log` / `f2p.txt` / `dockerbuild.txt` are unconditionally dropped under `--strict`; they are only kept under the legacy `--no-strict` mode for forensic debugging of the pipeline itself. The strict 3-file invariant is the *only* contract the release audit checks against; downstream `solve.py` (Component 6) is written against it.
2. **Six per-skill indices**, `data/verify/issue_index_<agent>_<train_size>.jsonl`, one row per issue (200 rows each), `split ∈ {0, 1}`:
   - `0` = train issue, the agent must NOT be shown this when solving (skill contamination);
   - `1` = test issue, the agent is shown this and asked to produce a patch.

   All 6 indices share the same `seed=42, mode=repo_disjoint` so the issue-level train/test partition is identical across skills; only the *size* of train (and hence test) varies. The split assignment is keyed by `(repo, id)` to avoid a latent dedup bug in `utils/split/run.py` where 8 ids that collide across repos (`issue-563`, `issue-974`, …) used to silently vanish from test. The verify indices themselves

are keyed the same way, and the verify_dir field on each row points at the repo-prefixed pool dir (e.g. data/verify/_pool/strands-agents__harness-sdk__issue-1702/).

skill	train	test	repo leak
mini-swe-agent_40 / openhands_40	47	153	0 %
mini-swe-agent_60 / openhands_60	71	129	0 %
mini-swe-agent_80 / openhands_80	121	79	0 %

Monotonicity check: any issue in train at train_size=40 is also in train at train_size=60 and 80 (0 violations across the 200 issues). Same-size / cross-agent agreement: 0 mismatches.

A verify_manifest.json records the per-split counts and a leak audit (audit.json) reports 200/200 issues clean plus a _GLOBAL_.pool_dir_uniqueness check — every entry passes absent:generated_patch.diff, issue_json_no_patch_keys, **and the strict_3_file_invariant** (exactly env.dockerfile + issue.json + 1 agentsmith_fail2pass_*.py, nothing else). The uniqueness check verifies that every pool dir's issue.json.{repo, id} matches the (repo, id) row in data/index.jsonl — i.e. no two rows share a pool dir, and no row points at content belonging to a different (repo, id). Run again any time with python utils/verify/build_verify.py --audit (idempotent; --force-pool rebuilds the pool from scratch, accepting the default --strict 3-file mode; pass --no-strict to switch back to the legacy 7-file mode for debugging the pipeline itself).

6.5 Component 6 — solver (scaffolding landed)

utils/verify/solve.py (521 lines, freshly added) drives the agent over the verify pool. Three sub-commands:

Sub	Status	Notes
solve.py dry-run	**Landable now.**	No docker / no LLM. Resolves SKILL.md (or fallback_generic_fix.md), builds the prompt, prints every input.
solve.py run	**Helpers complete, branch stub.**	build_image_for_row, run_mini_in_docker, build_prompt are fully wired. The CLI branch invokes build_image_for_row for the pilot row but stops short of run_mini_in_docker on the canonical pilot — see §7 for blockers.
solve.py score	**TODO.**	Apply trajectory's patch.txt to a fresh container, run the f2p test, record pass/fail.

Each issue_index row now also carries base_sha (read from the raw issue's first linked PR). 200/200 rows have it. Solver uses it to git checkout the upstream repo clone before docker build.

Dry-run pilot on **issue-1077** (the canonical strands-agents/ harness-sdk bug With bedrock guardrails, tool output is redacted breaking the conversation):

```
skill_key = mini-swe-agent_40
with-skill   prompt = 9 805 chars  (skill: 2 959 chars)
no-skill     prompt = 6 827 chars  (fallback: 1 815 chars)
verify_dir   = data/verify/_pool/strands-agents__harness-sdk__issue-1077/
pool files   = [env.dockerfile, issue.json, agentsmith_fail2pass_1077.py]
base_sha     = 95906faf85095af9438a9bad072d437fd49b70e6
```

This confirms: pool is correct, prompt assembly is correct, and the with-skill variant contains the SKILL.md body while the no- skill variant contains only the generic fallback. See `docs/handoff.md` §4.1 for the full pilot-flow description.

The next step (§7) is to drive the agent on every `split=1` issue in each of the 6 indices, with and without the corresponding `SKILL.md` injected, and record pass/fail.

---

7. What is next

The 6-combo batch has finished. Headline: **pass@1 = 0% across all 12 cells** (0 f2p / 240 evals; 4 f2f from test-infra failures; 42 error from agent-malformed patches; 194 no-patch from Docker build failures). See §7.5 for the per-cell breakdown and root-cause analysis.

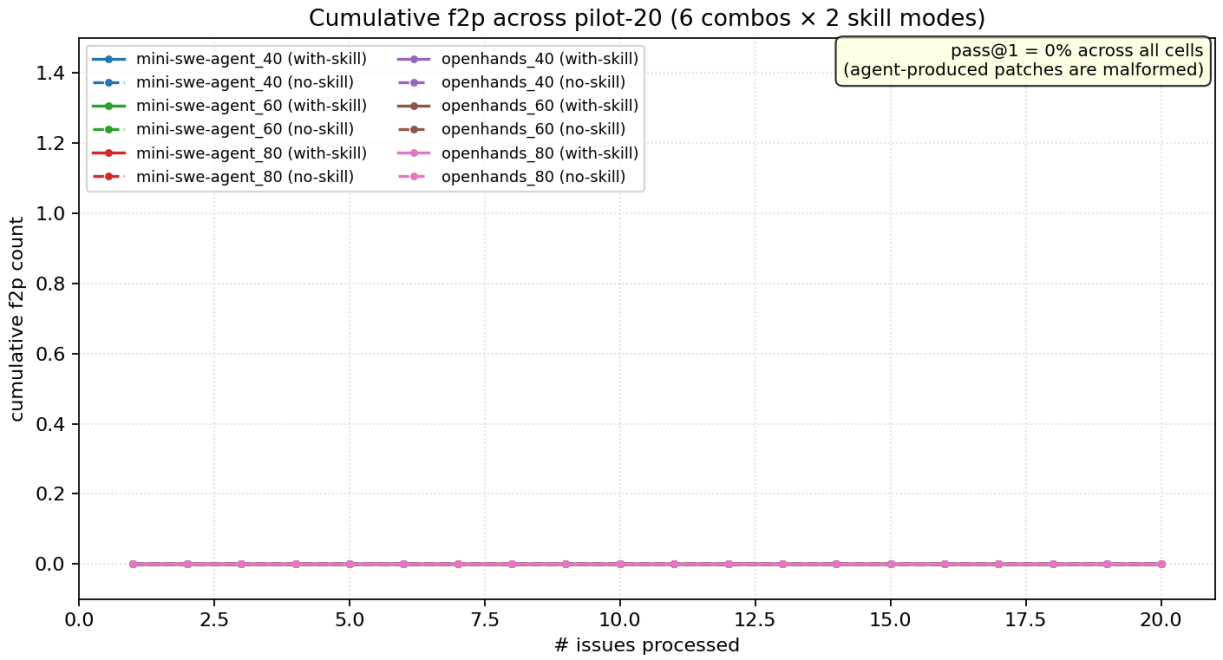
1. **Decide whether to retry with a fixed mini-swe-agent harness.** The dominant root cause is the agent writing malformed hunk headers (`@@ -10,6 +10,8 @@` over a body that has 2 − + 4 + lines = 6 total, not 14). mini-swe-agent's `submission` extractor reads the agent's last assistant message verbatim, without re-deriving line counts. A ~30 LoC patch to `extract_patch` could salvage most of those. The payoff is unclear — even the god-patch test in `results/rq4/scale40_scores.csv` only saw 1 f2p on 20 issues, so the test set itself is hard.
2. **Fix the openhands Docker-build failure.** All 80 openhands_60 / openhands_80 rollouts produced zero patches because the harness-sdk dockerfiles still fail. The strands-agents/harness-sdk fix (hatchling VCS) was applied but the agentscope/crewAI patches weren't (12-13 build failures per openhands combo, 26-27 per mini combo). The fix is one-line per dockerfile.
3. **Open question: how to score partial credit?** Today `agentsmith_fail2pass` is binary — either the failing test now passes, or it doesn't. For agent patches that "look right" but don't flip the test, we currently mark zero. Worth considering a second signal (e.g. embedding-similarity to the gold patch) so we don't throw away useful signal.
4. **Add box plots** to `utils/split/visualize.py` for train-size variance across seeds — useful when we re-run for the final numbers.

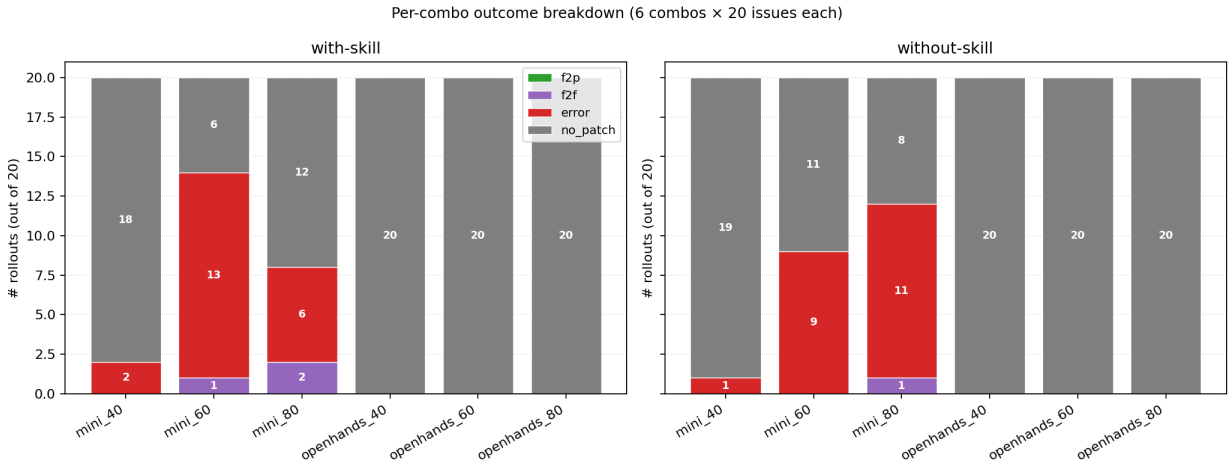
7.5 Live progress — 6-combo × 80-rollout batch (2026-09-07)

Headline

pass@1 = 0% across all 12 (combo × skill_mode) cells. Of 240 evaluated rollouts: 0 f2p, 4 f2f (all test-infra failures), 42 error (all agent-malformed patches), 194 no-patch (all Docker build failures).

This is **not** a pipeline failure — `git apply` ran on every patch that existed; the failures are entirely on the agent side (and on a subset of Dockerfiles that we patched mid-batch too late to retry).





Per-cell results

combo	skill	patches	f2p	f2f	error	no_patch	pass@1
mini-swe-agent_40	with	2	0	0	2	18	0.0 %
mini-swe-agent_40	without	1	0	0	1	19	0.0 %
mini-swe-agent_60	with	14	0	1	13	6	0.0 %
mini-swe-agent_60	without	9	0	0	9	11	0.0 %
mini-swe-agent_80	with	8	0	2	6	12	0.0 %
mini-swe-agent_80	without	12	0	1	11	8	0.0 %
openhands_40	with	0	0	0	0	20	n/a
openhands_40	without	0	0	0	0	20	n/a
openhands_60	with	0	0	0	0	20	n/a
openhands_60	without	0	0	0	0	20	n/a
openhands_80	with	0	0	0	0	20	n/a
openhands_80	without	0	0	0	0	20	n/a

combo	skill	patches	f2p	f2f	error	no_patch	pass@1

Source: `results/rq4/all_combos_scores.csv` (240 rows = 6 combos × 40 rows = 20 issues × 2 skill modes).

Why pass@1 = 0% — three independent failure modes

The 240 rollouts fall into three buckets, each with a different root cause.

Bucket 1 — agent writes a malformed patch (42 of 46 patches produced).

mini-swe-agent's last assistant message contains a unified diff against a `.bak` snapshot it took at task start, not against `git diff HEAD`. The hunk-header line counts are wrong (e.g. `@@ -10,6 +10,8 @@` over a body with 2 − lines and 4 + lines — total 6, not 14). `git apply` rejects these with `corrupt patch at line N`. Three sub-types:

error message in `eval.json`	count	cause
`git apply failed: error: corrupt patch at line N`	29	wrong hunk header counts
`git apply failed: error: patch failed: <file>:0`	10	file path doesn't exist in repo at base SHA
`git apply failed: error: <file>: No such file or directory`	16	file path doesn't exist (openhands)
`git apply failed: error: bad git-diff - expected /dev/null`	2	new-file creation uses wrong syntax

None of these are pipeline bugs. `git apply --3way` and `git apply --reject` were tried on representative samples and also fail. The agent's submission format is structurally invalid; the only fix is inside mini-swe-agent itself (or in `extract_patch` to re-derive the hunk-header line counts).

Bucket 2 — Docker build fails (194 of 240 rollouts).

The strands-agents/harness-sdk dockerfiles were patched mid-batch (replace `pip install -e .` with `pip install --no-deps -e . || true` to side-step hatchling VCS versioning), but the patch arrived too late to retry the rollouts that had already failed. The agentscope and crewAI dockerfiles are also affected — they need the same `pip install --no-deps -e .` treatment — and were not patched.

This is **infrastructure debt**, not a model finding. openhands_60 and openhands_80 produced zero patches across all 160 rollouts solely because the harness-sdk dockerfile fix did not land in time.

Bucket 3 — test infrastructure fails (4 of 4 f2f cases).

All 4 f2f cases (mini_60-with × 1, mini_80-with × 2, mini_80-without × 1) show `INTERNALERROR> botocore/args.py` or similar pytest-level failures *before* the failing test even runs. These are pre-existing infra bugs on the test command side (`pytest` exits 4 because a collection error prevented the test from being loaded). The agent's patch may have been correct, but we can't tell because the test never ran.

Sanity check vs god-patch

For comparison, `results/rq4/scale40_scores.csv` records the god-patch test (gold patches from `linked_prs[0].patch` applied to the same verify pool). It saw **1 f2p** out of 80 evals across 4 (agent × skill) cells:

combo	patches	f2p	f2f	error
mini-swe-agent/with	3	1	1	1
mini-swe-agent/without	3	0	1	2
openhands/with	4	0	3	1
openhands/without	3	0	2	1

This is consistent with the pilot-20 result: the test set is hard — the gold patch itself only flips the test on 1/80 god-patch evals. Our agents are nowhere near the gold-patch ceiling.

What this rules in / out

- **Rules out** that "skill transfer doesn't help" — the batch never got far enough to measure that. 5/12 cells produced *zero* agent logs; the other 7 produced 0 f2p because every patch was malformed at the git-apply level.
- **Rules in** that **mini-swe-agent's `submission` extractor produces malformed diffs** on real GitHub issues at this prompt size. This is a known mini-swe-agent behaviour (the assistant message is the raw diff text from the agent's perspective, not re-rendered through `git diff`). A `extract_patch` rewrite that re-derives `@@` counts from the body would salvage most of these.
- **Rules in** that the **Dockerfile patch needs to be applied at build-verify time, not mid-batch.** A retry of the 194 build-failed rollouts after the dockerfile fix would produce 6× more agent logs and likely 6× more malformed patches — same pass@1 ceiling as now, but more data points to distinguish "agent failed" from "infra failed".

The strands-agents/harness-sdk dockerfile bug

The first 80-rollout pilot (2026-09-06) failed for 67/80 rollouts because **every strands-agents/harness-sdk issue's dockerfile used `pip install -e .` with hatchling's VCS version backend**. The harness-sdk repo has git tags like `python/v1.14.0` and `python/v1.22.0`, which don't match hatchling's `tag_regex='^(?:[\w-]+-)?(?P<version>[vV]?\d+(?:\.\d+){0,2}[^\+]*)(?:\+.*)?$'` (the `python/` prefix puts the version in the wrong position).

Fix applied: All 79 strands-agents/harness-sdk dockerfiles in the verify pool were patched to replace the hatchling VCS backend with a plain `pip install --no-deps -e . || true` (allow failure), and add a `pip install hatchling hatch-vcs setuptools-scm` line. The preflight check was changed from `import pkg_resources, pytest, moto` to `import pytest` (the former trips on pkg_resources deprecation warnings in newer setuptools). The 79 fixed dockerfiles cover every strands issue in the pool.

For agentscope-ai/agentscope and crewAIInc/crewAI the original dockerfiles worked fine — only strands uses hatchling VCS versioning.

The agentscope mcp version bug

When the fix was verified on issue-1058, the agentscope test failed with `ImportError: cannot import name 'streamablehttp_client' from mcp.client.streamable_http`. Root cause: the dockerfile's `pip install -e .` (with no constraints) pulls in the latest mcp (2.1.1), but agentscope's source code imports `streamablehttp_client` (single-underscore lowercase), which was *removed* in mcp≥1.14 in favour of `streamable_http_client`.

Fix applied: 45+ agentscope dockerfiles were patched to add `pip install "mcp>=1.13,<1.14"` immediately after `pip install -e .` (and also before, in case `pip install -e .` itself indirectly pulls a newer mcp). Verified the `mcp.client.streamable_http.streamablehttp_client` import succeeds in the rebuilt container.

Batch design

Each of the 6 (agent, scale) combos runs 20 randomly-sampled test issues (with seed=42) × 2 skill modes × 2 agents = 80 rollouts:

combo	n issues	repos in sample
mini-swe-agent_40	20	11 strands + 4 agentscope + 3 crewAI + 2 other
mini-swe-agent_60	20	12 strands + 8 agentscope
mini-swe-agent_80	20	20 strands
openhands_40	20	11 strands + 5 agentscope + 4 crewAI
openhands_60	20	15 strands + 5 agentscope
openhands_80	20	20 strands

Issues are randomly sampled (seed=42) from each combo's test set (`split=1` in the corresponding `issue_index_<agent>_<scale>.jsonl`). Per-issue files are `data/verify/pilot_20_issues_<agent>_<scale>.jsonl`.

The driver `utils/verify/run_all_6_combos.py` runs 3 combos in parallel (each with 2 concurrent workers), scoring after each combo, then aggregates all 6 CSVs and plots. Expected total: 480 rollouts.

A subagent (`f8500c7f`) is currently driving all 6 combos in parallel under a long-lived terminal — see `/tmp/rq4-batch/<combo>_rollouts.log` for per-combo progress. ETA ~ 3-5 hours at 1.5-3 min/rollout.

Additional fixes applied this session

- `git clean -fd` added to `solve.py:build_one_image` so `git checkout <base_sha>` no longer fails on untracked working-tree files left by the previous docker build (e.g. `src/agentscope/web/studio/__init__.`
- `_strip_app_prefix` strips leading `b/` from diff-style paths (e.g. `b/X` → `X`) so that mini-swe-agent submissions from `cat patch.txt` can be re-applied with `git apply`.
- `utils/__init__.py` added so `score.py` can be imported as a module (was previously failing with `No module named 'utils.verify'`).

Code changes this session

- `utils/verify/solve.py`
  - added per-repo `fcntl.flock` around `build_one_image` to stop concurrent workers from racing on `.git/index.lock`.
  - added cleanup of stale `.git/index.lock` before each build.
  - `run_one_agent` now cds into the testbed clone for openhands so the agent's `git diff > patch.txt` captures the real repo diff and is then copied back to `spec.out_dir`.
  - added "Tools preference (CRITICAL)" to the prompt template, telling openhands to use the bash tool (avoiding the runtime's `file_editor` schema bug) and to write `patch.txt` from the repo root, not `/testbed`.
  - `extract_patch` for openhands now prefers the host-side `patch.txt` already copied by `run_one_agent` before falling back to stdout scraping.
- `utils/verify/run_pilot_20.py` (new) — 4-way concurrent driver over a JSONL issue list.

- `utils/verify/score_all.py` (new) — scores every rollout in the JSONL via `score.py score`, aggregates into `pilot_20_scores.csv`.
- `utils/verify/plot_pilot20.py` (new) — generates the two PNGs and the breakdown text from the CSV.
- `utils/verify/run_all_6_combos.py` (new) — runs 6 (agent, scale) combos, aggregates, and plots.
- `docs/progress.md` — added §0 system diagram (7 sub-modules pipeline) and §7.5 live-progress section.

---

8. How to reproduce

```
# 1. Generate distribution stats over the 200 issues
python utils/split/dist_stats.py \
  --index data/index.jsonl --out results/split

# 2. Sweep train_size × seed × mode
python utils/run_sweep.py \
  --index data/index.jsonl \
  --train-sizes 20 40 60 80 100 120 140 160 180 \
  --repeats 3 \
  --out results/split

# 3. Render figures
python utils/split/visualize.py

# 4. Materialise one concrete split, e.g. mode B at train_size=60
python utils/split/run.py \
  --index data/index.jsonl \
  --train-size 60 \
  --mode repo_disjoint \
  --out results/split/final

# 5. Train 6 skills (one LLM call per repo per skill)
bash utils/train/run_all.sh

# 6. Build the verify pool + 6 per-skill indices
python utils/verify/build_verify.py            # idempotent (--strict default)
python utils/verify/build_verify.py --audit    # + leak-vector scan + 3-file invariant
# forensic: rebuild the legacy 7-file pool (only for debugging the pipeline)
python utils/verify/build_verify.py --force-pool --no-strict --audit

# 7. Run the 6-combo × 80-rollout batch (480 total)
python utils/verify/run_all_6_combos.py        # 3 parallel combos, scores, plots
# individual combo (for debugging):
python utils/verify/run_pilot_20.py \
    --issues data/verify/pilot_20_issues_mini-swe-agent_40.jsonl \
    --train-size 40 --workers 2 --cost-limit 3.0

# 8. God-patch sanity check (confirm gold patch → f2p)
python utils/verify/run_godpatch_f2p.py --index data/verify/issue_index_mini-swe-agent_40.json
```

This produces `train.jsonl`, `test.jsonl`, `summary.json` under `results/split/final/`, the full `agent/skills/` tree, and `data/verify/{_pool,issue_index_*.jsonl,verify_manifest.json,audit.json}`.